

Practical : - 9

Aim : - To interface Fingerprint Sensor (R307S) with Raspberry Pi.

A fingerprint sensor is a biometric electronic device that captures a digital image or pattern of the unique ridges and valleys on a person's finger to identify or verify their identity.

How It Works

- **Capture:** The sensor takes an impression or picture of your fingerprint using light, electricity, or sound waves.
- **Extract:** Software finds special markers called minutiae, such as where lines end or split.
- **Compare:** It turns these markers into a unique mathematical template and compares it to a saved version.
- **Authenticate:** If the patterns match, access is granted.

Part I : - Connection

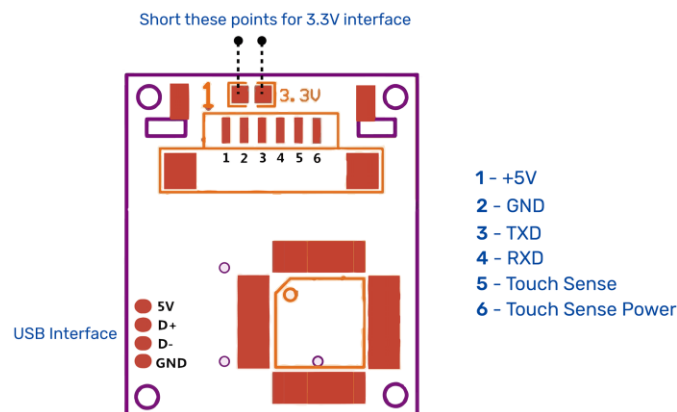
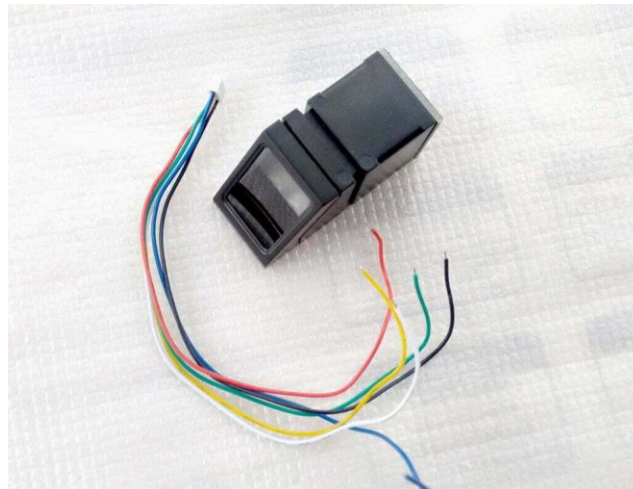


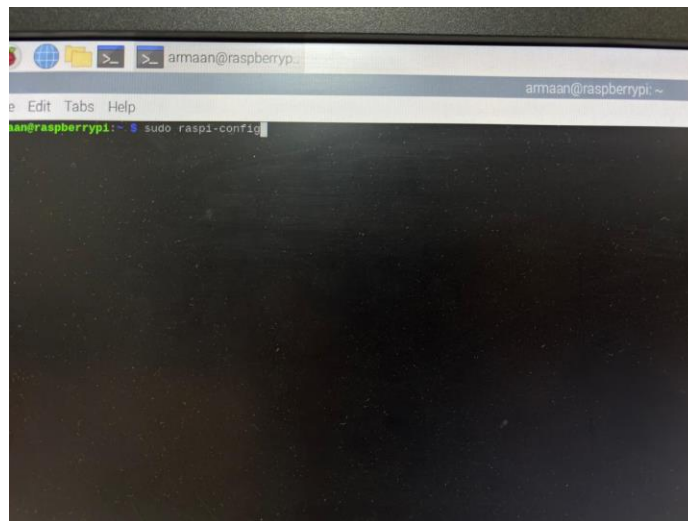
Fig. R307 Fingerprint Scanner Pinout

Pin No. 1 Fingerprint Sensor(VCC) = Pin no. 2 of Raspberry Pi (5V)
Pin No. 2 Fingerprint Sensor (GND) = Pin no. 6 of Raspberry Pi (GND)
Pin No. 3 Fingerprint Sensor (TXD) = Pin no. 10 of Raspberry Pi (RXD)
Pin No. 4 Fingerprint Sensor (RXD) = Pin no. 8 of Raspberry Pi (TXD)
Pin No. 5 Fingerprint Sensor (Touch Sense) = No connection (X)
Pin No. 6 Fingerprint Sensor (Touch Sense Power) = No connection (X)

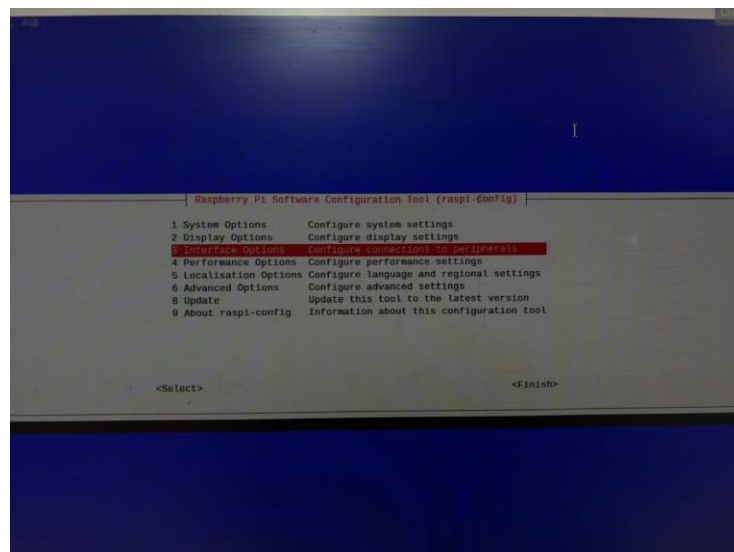
Part – II : - Library Installation

1. Enable UART on the Raspberry Pi.

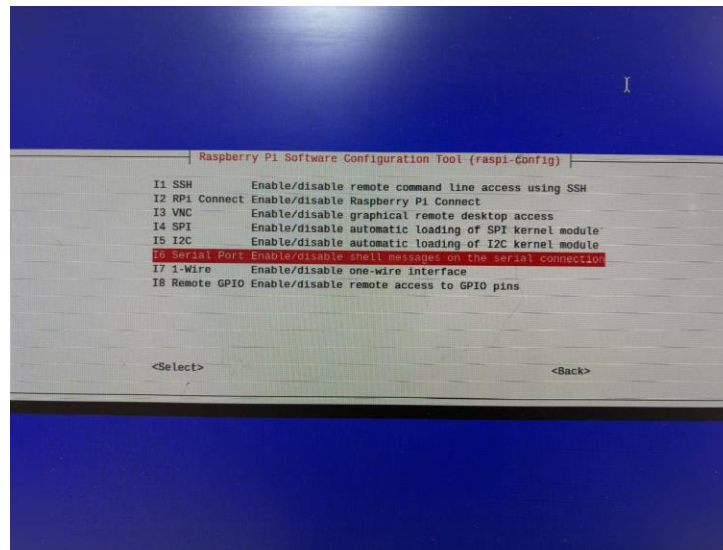
- **sudo raspi-config**



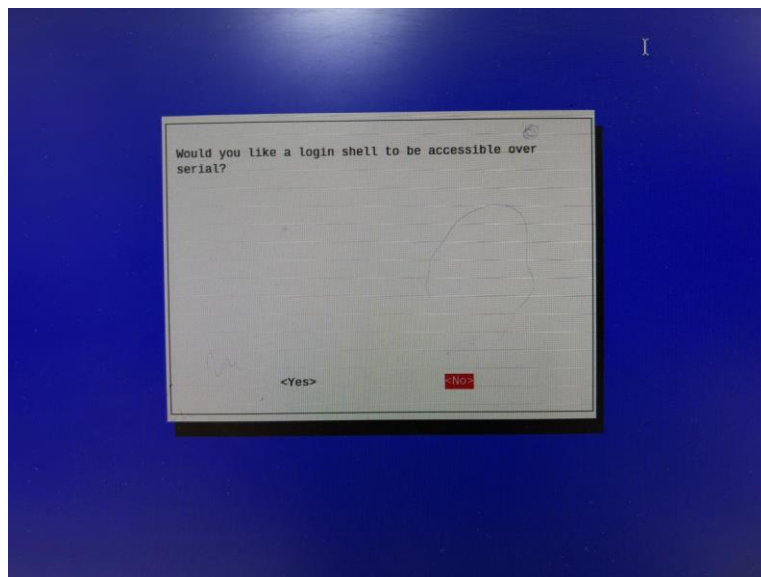
Go to Interface Options →



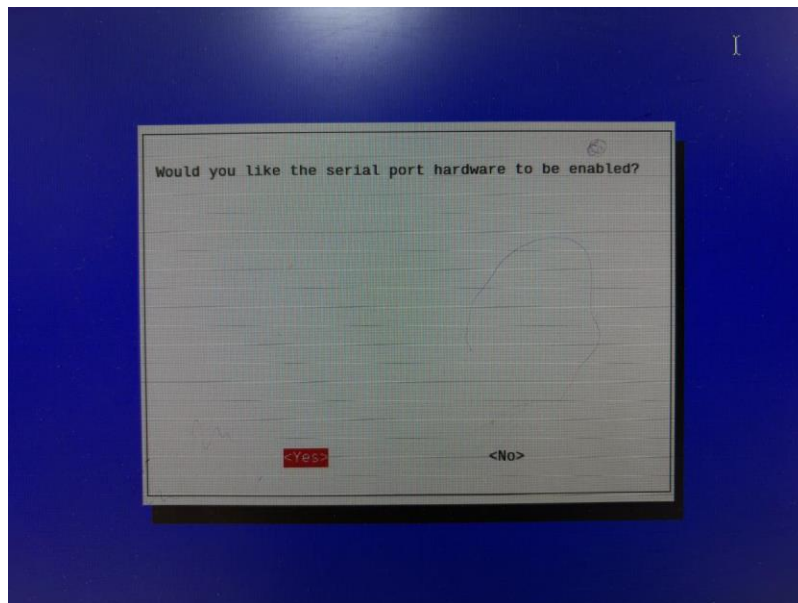
Go to Serial Port →



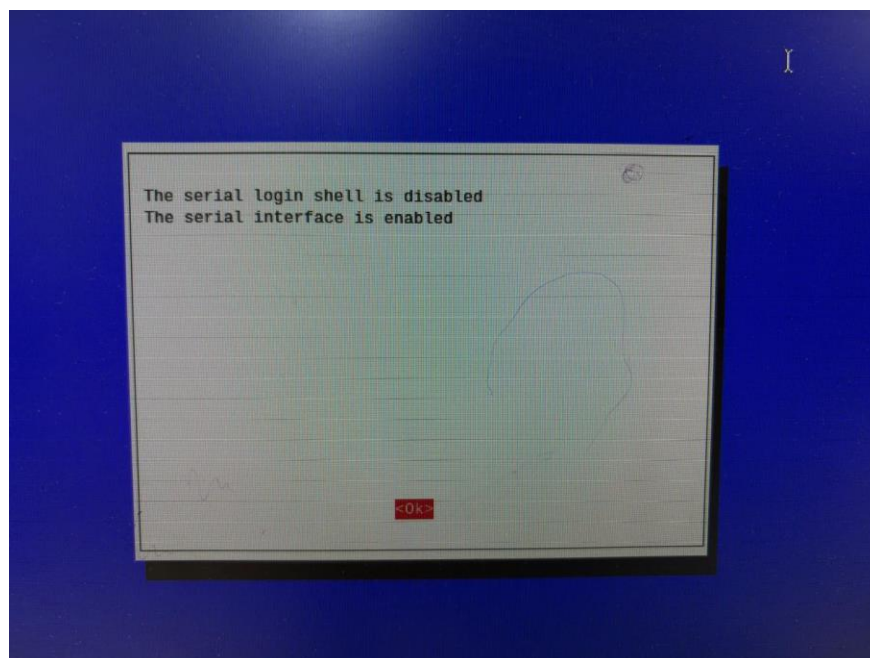
Login shell over serial → No

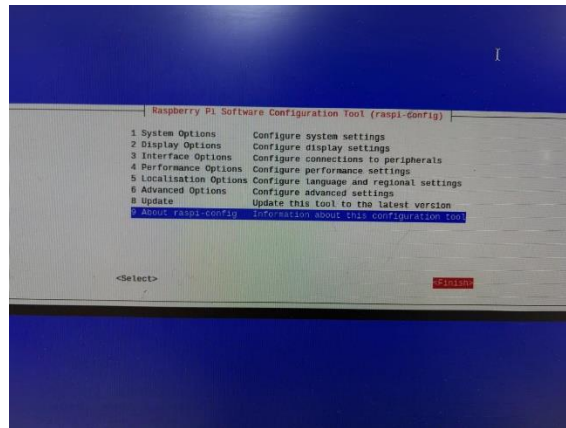


Serial Port Hardware → Yes

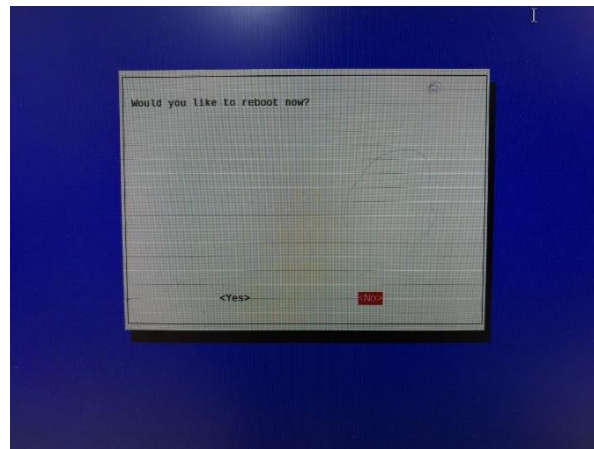


Press OK →

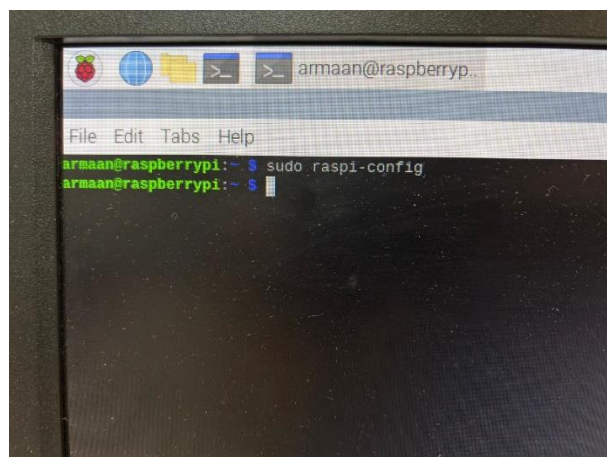




Reboot → No



Comeback To Main Terminal →

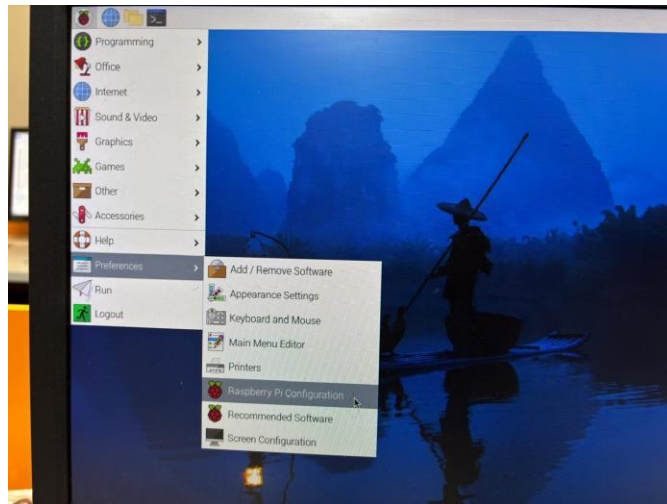


Close the terminal and come to desktop :-

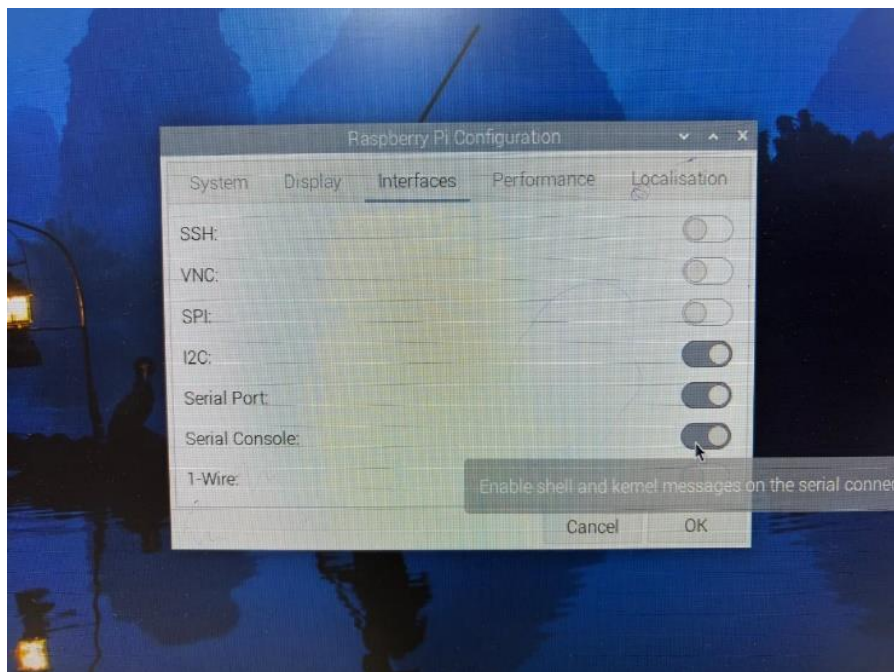
2. To check the status of Serial Console :-

Click on Raspberry Pi logo on the top left of your screen

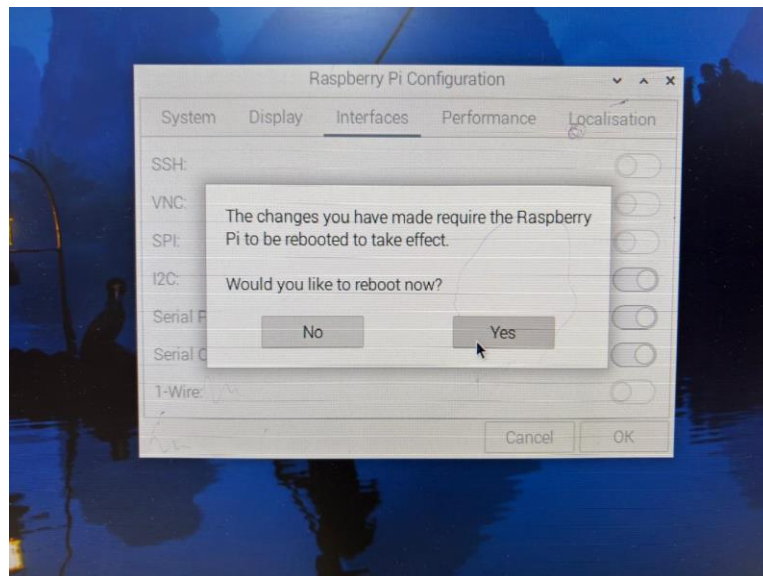
Got to Preferences → Raspberry Pi Configuration



Click on interface → Make Serial Port and Serial Console Enable



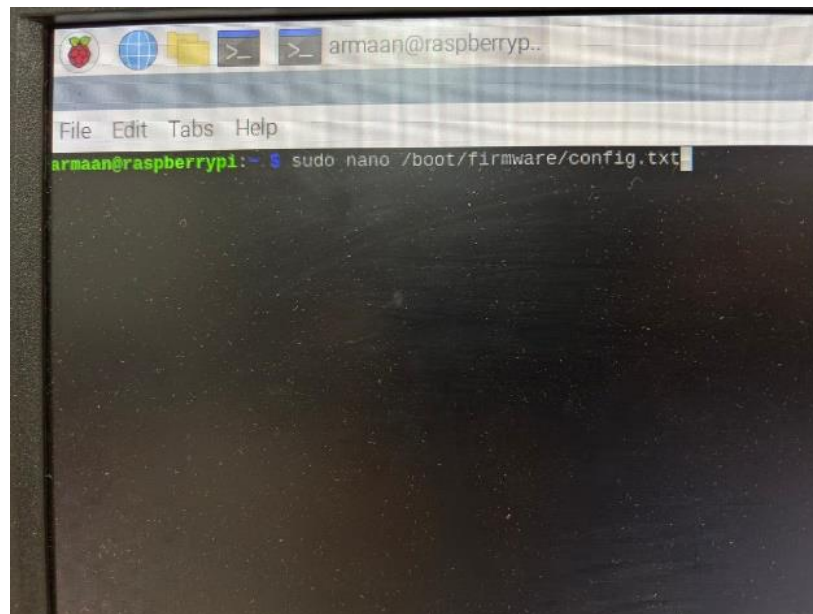
Click on OK → Reboot → Yes



3. Check config.txt

Open the terminal →

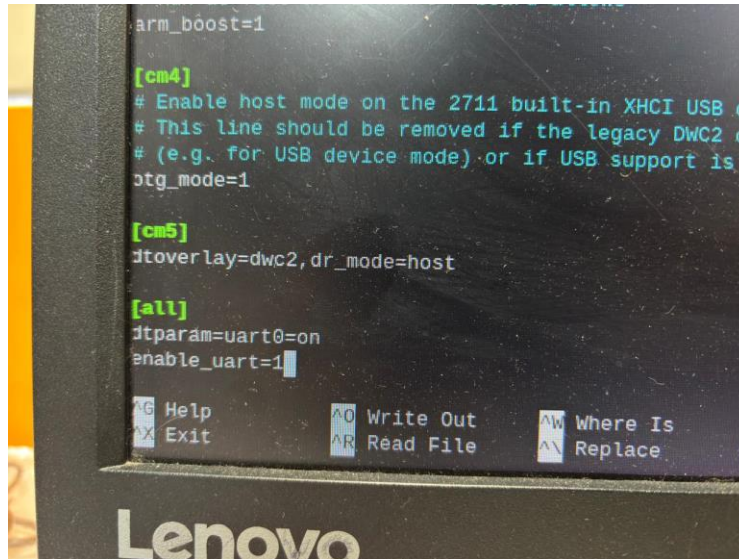
- **sudo nano /boot/firmware/config.txt**



Go at the end and check → `dtparam=uart0=on` exists

Type →

- `enable_uart=1`



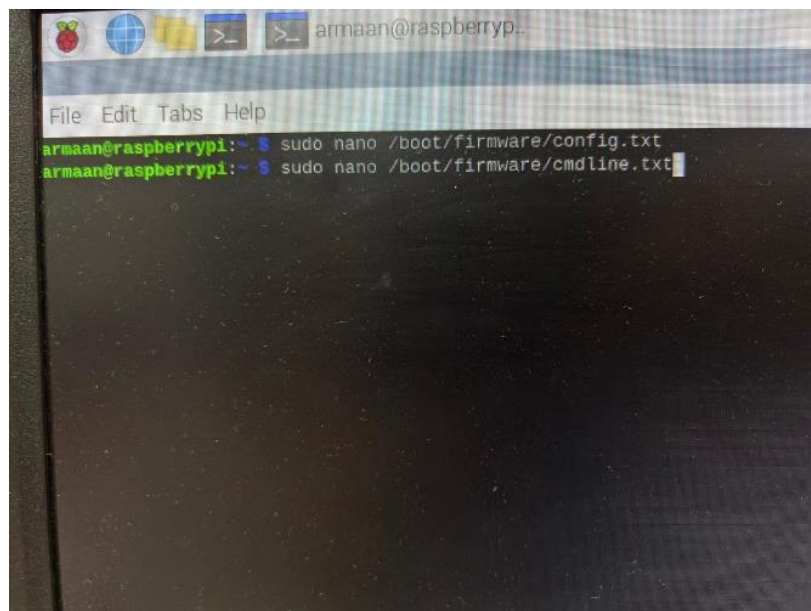
Press→

- `Ctrl+O` (to save)
- `Enter`
- `Ctrl+X` (to exit)

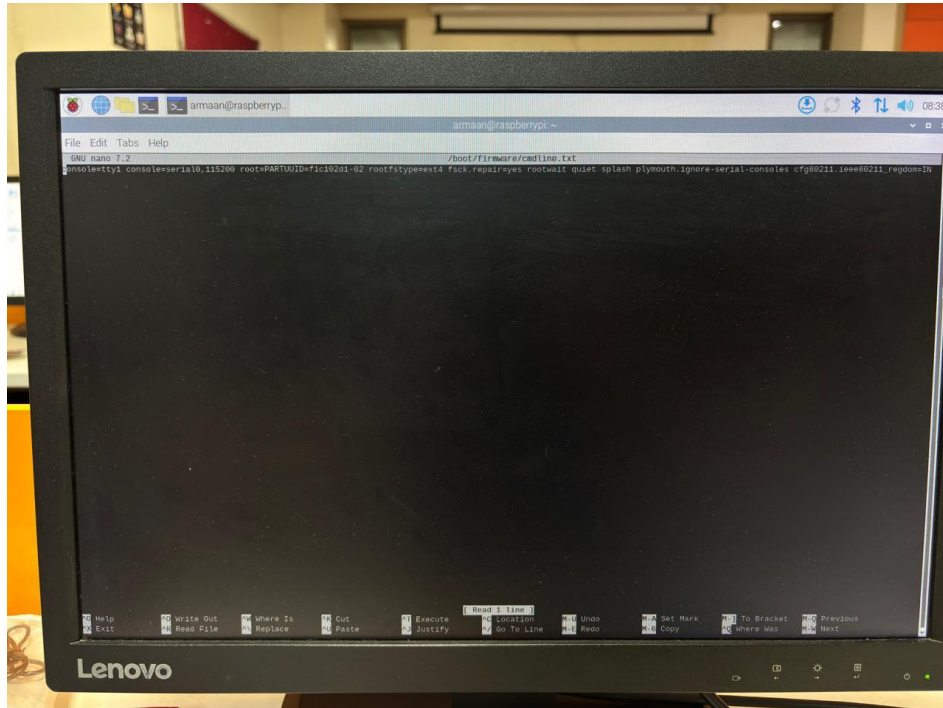
4. Remove the serial console from cmdline.txt (Critical Step)

Go to terminal and Type →

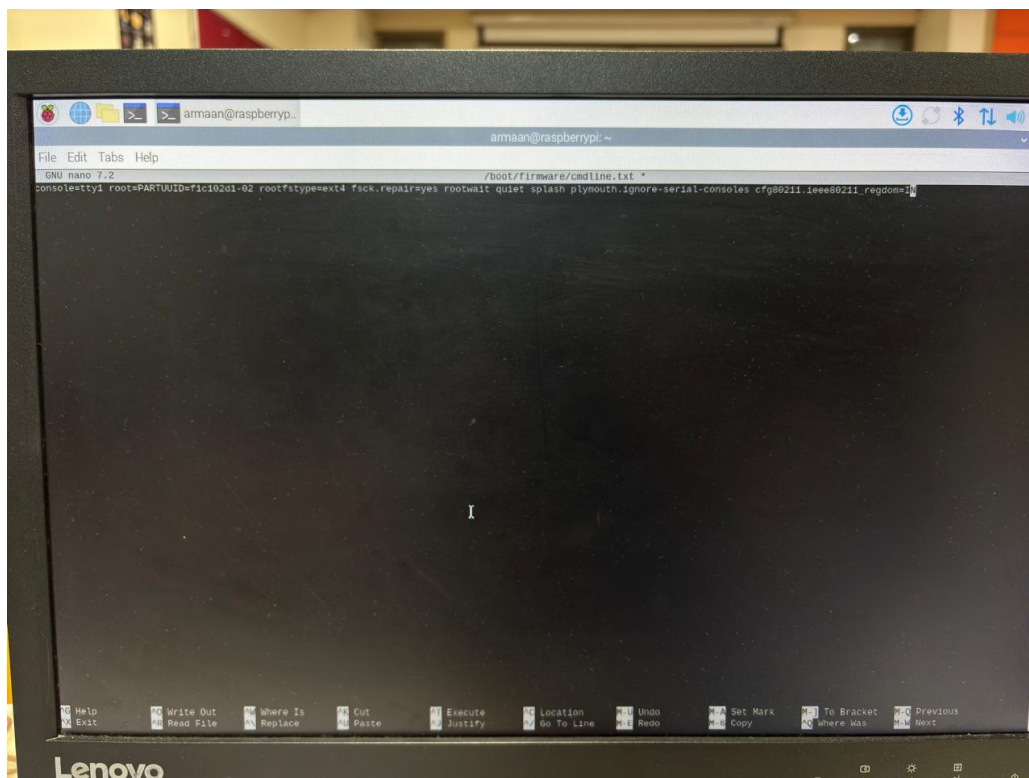
- `sudo nano /boot/firmware/cmdline.txt`



Remove only → console=serial0,115200
(Use Cursor to move and Backspace to delete)



So that it becomes →



Press → Ctrl+O → Enter → Ctrl+X

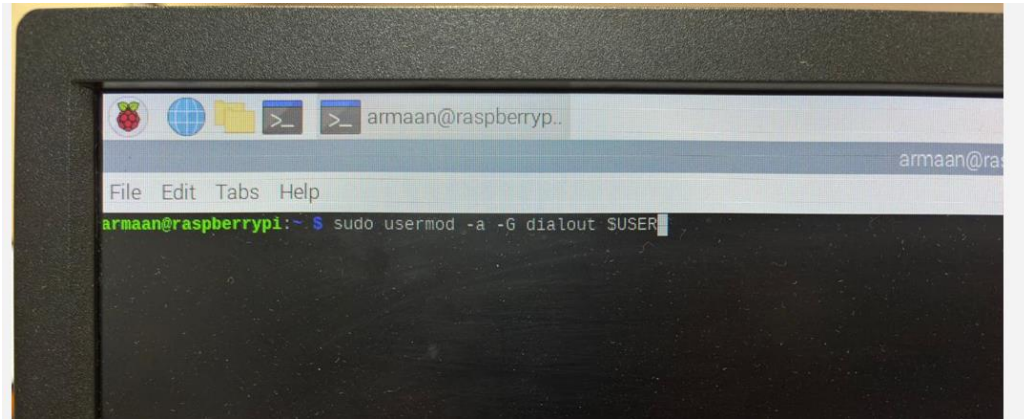
We'll come back to the main terminal

Save and Reboot :

- **sudo reboot**

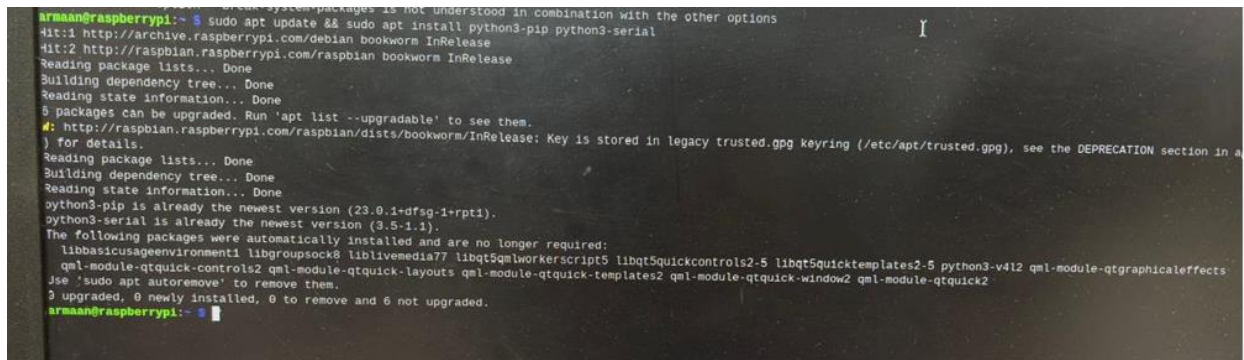
5. Add your user to the dialout group

- **sudo usermod -a -G dialout \$USER**

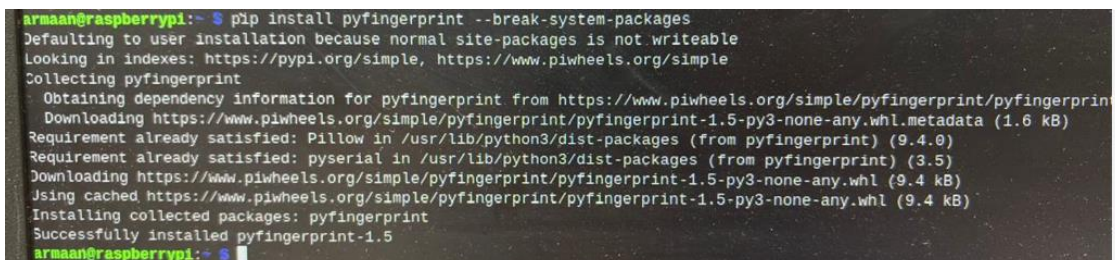


6. Install the Python and the fingerprint library

- **sudo apt update && sudo apt install python3-pip python3-serial**



- **pip install pyserial adafruit-circuitpython-fingerprint --break-system-packages**



Part III : - Code

Fingerprint_Controller.py

```
#!/usr/bin/env python3
"""
R307S Fingerprint Attendance/ID Tool
-----
All-in-one script:
1) Enroll a new fingerprint (with a name attached)
2) Verify a fingerprint -> shows whose finger it is
3) List everyone enrolled so far
4) Exit

Names are not stored on the sensor itself (the sensor only stores
fingerprint templates against numeric IDs), so this script keeps a
small local JSON file (fingerprint_names.json) in the same folder,
mapping each ID to a name.

Requirements:
pip install pyfingerprint --break-system-packages

Wiring (R307S is 3.3V TTL, no level shifter needed):
Sensor VCC (red) -> Pi Pin 2 (5V)
Sensor GND (black) -> Pi Pin 6 (GND)
Sensor RX (white) -> Pi Pin 8 (GPIO14 / TXD)
Sensor TX (yellow) -> Pi Pin 10 (GPIO15 / RXD)
"""

import sys
import time
import json
import os

try:
    from pyfingerprint.pyfingerprint import PyFingerprint
except ImportError:
    print("pyfingerprint is not installed. Run: pip install pyfingerprint --break-system-packages")
    sys.exit(1)

# ---- Config ----
SERIAL_PORT = '/dev/serial0' # try '/dev/ttyAMA0' if this doesn't work
BAUD_RATE = 57600
PASSWORD = 0x00000000 # R307S's actual default password

NAMES_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'fingerprint_names.json')

# ----- Name mapping (JSON file) -----
```

```

def load_names():
    if not os.path.exists(NAMES_FILE):
        return {}
    try:
        with open(NAMES_FILE, 'r') as fh:
            return json.load(fh)
    except (json.JSONDecodeError, IOError):
        return {}

def save_names(names):
    with open(NAMES_FILE, 'w') as fh:
        json.dump(names, fh, indent=2)

# ----- Sensor connection -----

def flush_stale_bytes():
    """Clears any leftover bytes sitting in the serial buffer."""
    try:
        import serial
        raw = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(0.3)
        raw.reset_input_buffer()
        raw.reset_output_buffer()
        raw.close()
        time.sleep(0.3)
    except Exception:
        pass

def connect():
    """Connects to the sensor and prints basic info."""
    flush_stale_bytes()

    last_error = None
    for attempt in range(1, 4):
        try:
            f = PyFingerprint(SERIAL_PORT, BAUD_RATE, 0xFFFFFFFF, PASSWORD)
        except Exception as e:
            last_error = f"connect error: {e}"
            time.sleep(0.5)
            continue

        if f.verifyPassword():
            print("[OK] Connected to the module successfully!")
            print(f" Capacity      : {f.getStorageCapacity()} templates")
            print(f" Stored templates: {f.getTemplateCount()}")
            return f
        else:
            last_error = "password verification returned false"

```

```

        print(f'[RETRY {attempt}/3] Password verification failed, retrying...')
        time.sleep(0.5)

    print(f'[ERROR] Could not connect after 3 attempts. Last reason: {last_error}')
    sys.exit(1)

# ----- Enroll -----

def enroll(f, names):
    try:
        print("\nPlace your finger on the sensor...")
        while not f.readImage():
            pass
        f.convertImage(0x01)

        result = f.searchTemplate()
        position_number = result[0]
        if position_number >= 0:
            existing_name = names.get(str(position_number), "(name not on record)")
            print(f'[INFO] This finger is already enrolled — {existing_name} (ID #{position_number})')
            return

        name = input("Enter a name for this finger: ").strip()
        if not name:
            print("[ERROR] Name cannot be empty, enrollment cancelled.")
            return

        print("Remove your finger...")
        time.sleep(2)

        print("Place the same finger again...")
        while not f.readImage():
            pass
        f.convertImage(0x02)

        if f.compareCharacteristics() == 0:
            print("[ERROR] The two scans didn't match, try again.")
            return

        f.createTemplate()
        position_number = f.storeTemplate()

        names[str(position_number)] = name
        save_names(names)

        print(f'[OK] Enrolled {name}'s finger! (ID #{position_number})')

    except Exception as e:
        print(f'[ERROR] Enrollment failed: {e}')

```

```
# ----- Verify -----
```

```
def verify(f, names):
```

```
    try:
```

```
        print("\nPlace your finger on the sensor to verify...")
```

```
        while not f.readImage():
```

```
            pass
```

```
        f.convertImage(0x01)
```

```
        result = f.searchTemplate()
```

```
        position_number, accuracy_score = result
```

```
        if position_number == -1:
```

```
            print("[NO MATCH] This finger doesn't match any enrolled ID.")
```

```
        else:
```

```
            name = names.get(str(position_number), "(name not on record for this ID)")
```

```
            print(f"\n>>> Welcome, {name}! <<<")
```

```
            print(f"    ID #{position_number}, confidence score: {accuracy_score}")
```

```
    except Exception as e:
```

```
        print(f"[ERROR] Verification failed: {e}")
```

```
# ----- List -----
```

```
def list_enrolled(names):
```

```
    if not names:
```

```
        print("\nNo one is enrolled yet.")
```

```
        return
```

```
    print("\n=== Enrolled people ===")
```

```
    for id_str, name in sorted(names.items(), key=lambda x: int(x[0])):
```

```
        print(f"    ID #{id_str}: {name}")
```

```
# ----- Main -----
```

```
def main():
```

```
    print("=== R307S Fingerprint Attendance/ID Tool ===\n")
```

```
    f = connect()
```

```
    names = load_names()
```

```
    while True:
```

```
        print("\nWhat would you like to do?")
```

```
        print(" 1) Enroll a new fingerprint (with a name)")
```

```
        print(" 2) Verify a fingerprint (shows the name)")
```

```
        print(" 3) List all enrolled people")
```

```
        print(" 4) Exit")
```

```
        choice = input("Choice (1-4): ").strip()
```

```
        if choice == '1':
```

```

        enroll(f, names)
    elif choice == '2':
        verify(f, names)
    elif choice == '3':
        list_enrolled(names)
    elif choice == '4':
        print("Bye!")
        break
    else:
        print("Invalid choice, try again.")

if __name__ == '__main__':
    main()

```

Fingerprint_uart_diagnostic.py

```

#!/usr/bin/env python3
"""
R307S raw UART diagnostic tool
-----
pyfingerprint library ko bypass karke seedha serial port pe
'verify password' command bhejta hai aur RAW bytes dikhata hai.

Isse pata chalega:
- Kuch response aa raha hai ya bilkul nahi (wiring/power issue)
- Sahi header (EF 01) aa raha hai ya garbage (console conflict / baud mismatch)

Requirements:
    pip install pyserial --break-system-packages
"""

import serial
import time
import sys

PORT = '/dev/serial0' # agar kaam na kare to '/dev/ttyAMA0' try karo
BAUDS_TO_TRY = [57600, 9600, 19200, 38400, 115200]

def build_verify_password_packet():
    """VerifyPassword command packet banata hai (address FFFFFFFF, password FFFFFFFF)."""
    header = [0xEF, 0x01]
    address = [0xFF, 0xFF, 0xFF, 0xFF]
    pid = 0x01
    instruction = 0x13
    password = [0xFF, 0xFF, 0xFF, 0xFF]

```

```

length_bytes_count = 1 + 4 + 2 # instruction + password + checksum
length = [(length_bytes_count >> 8) & 0xFF, length_bytes_count & 0xFF]

checksum_data = [pid] + length + [instruction] + password
checksum = sum(checksum_data) & 0xFFFF
checksum_bytes = [(checksum >> 8) & 0xFF, checksum & 0xFF]

packet = header + address + [pid] + length + [instruction] + password + checksum_bytes
return bytes(packet)

def hex_dump(data):
    if not data:
        return "(kuch bhi nahi mila — empty response)"
    return ''.join(f'{b:02X}' for b in data)

def test_baud(port, baud):
    print(f"\n--- Testing baud rate: {baud} ---")
    try:
        ser = serial.Serial(port, baud, timeout=1)
    except Exception as e:
        print(f"[ERROR] Port kholte time fail: {e}")
        return False

    time.sleep(0.5)
    ser.reset_input_buffer()

    packet = build_verify_password_packet()
    print(f"Bhej raha hu : {hex_dump(packet)}")
    ser.write(packet)
    time.sleep(0.3)

    response = ser.read(64)
    print(f"Mila response: {hex_dump(response)}")

    ser.close()

    if len(response) >= 2 and response[0] == 0xEF and response[1] == 0x01:
        print(f"[OK] Sahi header (EF 01) mila is baud rate pe: {baud}")
        return True
    elif len(response) == 0:
        print(f"[NO RESPONSE] Kuch bhi wapas nahi aaya — wiring ya power check karo.")
        return False
    else:
        print(f"[GARBAGE] Kuch aaya but header galat hai — console conflict ya baud mismatch ho sakta hai.")
        return False

def reliability_loop(port, baud, rounds=5):

```

```

"""Same baud rate ko baar-baar test karke reliability check karta hai."""
print(f"\n=== Reliability check: {rounds} baar {baud} baud pe test kar raha hu ===")
ok_count = 0
for i in range(1, rounds + 1):
    print(f"\n[Round {i}/{rounds}]")
    if test_baud(port, baud):
        ok_count += 1
    time.sleep(1)

print(f"\n=== Reliability Result: {ok_count}/{rounds} successful ===")
if ok_count == rounds:
    print("Consistent hai — sensor/wiring/power sab theek hai. Ye software/library ka issue hai.")
elif ok_count == 0:
    print("Ek baar bhi success nahi hua — hardware/wiring/power issue hai.")
else:
    print("Kabhi kabhi kaam karta hai, kabhi nahi — loose connection ya marginal power ka signal hai.")

```

```

def main():
    print("=== R307S Raw UART Diagnostic ===")
    print(f"Port: {PORT}\n")

    success = False
    working_baud = None
    for baud in BAUDS_TO_TRY:
        if test_baud(PORT, baud):
            success = True
            working_baud = baud
            break

    print("\n=== Result ===")
    if success:
        print("Sensor sahi respond kar raha hai. fingerprint_test.py me yehi baud rate daal do.")
        reliability_loop(PORT, working_baud)
    else:
        print("Kisi bhi baud rate pe sahi header nahi mila. Ye check karo:")
        print(" 1. cat /boot/firmware/cmdline.txt me console=serial0 to nahi hai")
        print(" 2. TX aur RX wires swap karke dobara try karo")
        print(" 3. VCC-GND ke beech multimeter se voltage check karo (4.5V+ hona chahiye)")
        print(" 4. /dev/ttyAMA0 try karo PORT variable badal ke")

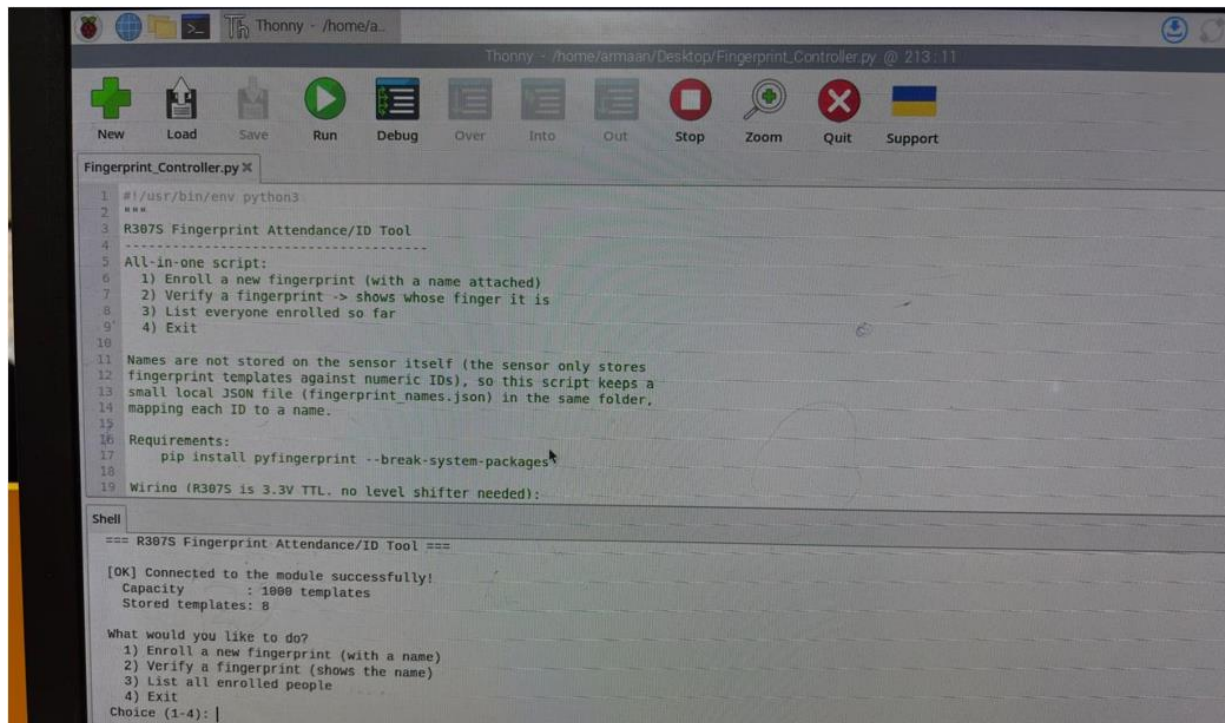
```

```

if __name__ == '__main__':
    main()

```

Part IV : - Output

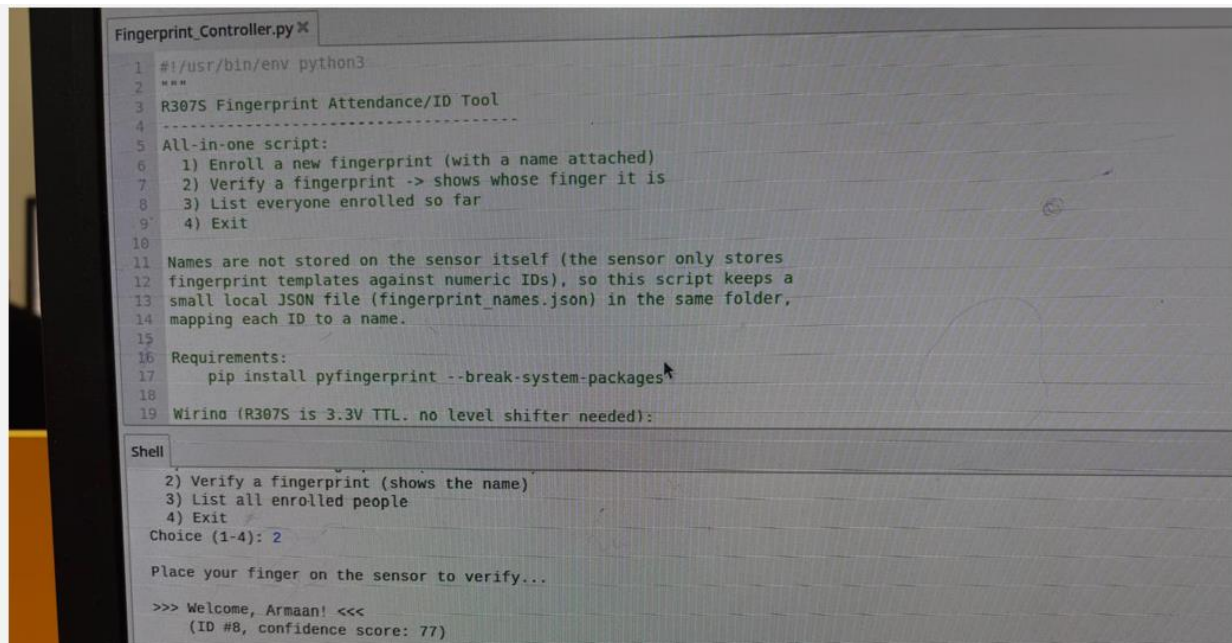


```
Thonny - /home/a...
Thonny - /home/armaan/Desktop/Fingerprint_Controller.py @ 213 : 11

New Load Save Run Debug Over Into Out Stop Zoom Quit Support

Fingerprint_Controller.py ✕
1 #!/usr/bin/env python3
2 """
3 R307S Fingerprint Attendance/ID Tool
4 -----
5 All-in-one script:
6 1) Enroll a new fingerprint (with a name attached)
7 2) Verify a fingerprint -> shows whose finger it is
8 3) List everyone enrolled so far
9 4) Exit
10
11 Names are not stored on the sensor itself (the sensor only stores
12 fingerprint templates against numeric IDs), so this script keeps a
13 small local JSON file (fingerprint_names.json) in the same folder,
14 mapping each ID to a name.
15
16 Requirements:
17     pip install pyfingerprint --break-system-packages
18
19 Wirina (R307S is 3.3V TTL, no level shifter needed):
20
Shell
=== R307S Fingerprint Attendance/ID Tool ===
[OK] Connected to the module successfully!
Capacity      : 1000 templates
Stored templates: 8

What would you like to do?
1) Enroll a new fingerprint (with a name)
2) Verify a fingerprint (shows the name)
3) List all enrolled people
4) Exit
Choice (1-4): |
```



```
Fingerprint_Controller.py ✕
1 #!/usr/bin/env python3
2 """
3 R307S Fingerprint Attendance/ID Tool
4 -----
5 All-in-one script:
6 1) Enroll a new fingerprint (with a name attached)
7 2) Verify a fingerprint -> shows whose finger it is
8 3) List everyone enrolled so far
9 4) Exit
10
11 Names are not stored on the sensor itself (the sensor only stores
12 fingerprint templates against numeric IDs), so this script keeps a
13 small local JSON file (fingerprint_names.json) in the same folder,
14 mapping each ID to a name.
15
16 Requirements:
17     pip install pyfingerprint --break-system-packages
18
19 Wirina (R307S is 3.3V TTL, no level shifter needed):
20
Shell
2) Verify a fingerprint (shows the name)
3) List all enrolled people
4) Exit
Choice (1-4): 2

Place your finger on the sensor to verify...

>>> Welcome, Armaan! <<<
      (ID #8, confidence score: 77)
```